Be Parallel, Be Cautious

With great power comes great responsibility. Elixir is a powerhouse when it comes to running parallel processes. You can literally spin up thousands of concurrent requests that are active at the same time. But this can cause problems.

Programs don't exist in a vacuum. They interact with other programs like databases or external APIs. A program can only handle so many parallel requests before it becomes overwhelmed. You could easily exhaust a database connection pool or downstream API service by sending too many simultaneous requests.

It's best to always have control over the amount of parallelism in your system. Libraries that you'll see later in this book, like Ecto and Oban, all include a limited number of parallel processes so that you're less likely to overwhelm your system. You can raise the limits if you have a system that can handle it, or you can decrease them to deal with a resource-constrained system.

Built-in libraries like Task[6] and functions like `spawn/1` will let you create one-time processes for parallel execution. This often ends badly if done in an uncontrolled way because it's easy to accidentally create too many simultaneous processes. Instead, stick to plain GenServers or build a data pipeline with a library like Oban to maintain control of your application's parallelism.

I wrote about creating a stable data pipeline in the "Avoid Performance Pitfalls" chapter of *Real-Time Phoenix [Bus20]*, but if you use Oban then you'll be in a good place.

We're going to switch gears to a more conceptual topic. You've learned about processes and established foundations to use them, but this leaves a big question remaining. What does the BEAM's process model allow us to do that we can't do in other programming languages? Let's tackle this next.

Build Cities, Not Skyscrapers

The traditional model of building applications—especially web applications—is single-stack-oriented. This means that applications work from an entry point (web request, CLI command, background worker, and so on) and execute code in a single stack or process to achieve their purpose. This programming model is clearly the dominant one in the Ruby community.

6. https://hexdocs.pm/elixir/main/Task.html